Option 3. Planning delivery routes

Decision owner and action

A dispatcher assigns customer deliveries to vehicles and chooses the order of stops on each route. The tool chooses which vehicle serves each customer and the stop order of each vehicle's route, and it reports the travel, waiting, and service consequences.

Why this matters

Distributors, grocery delivery services, and field-service companies plan routes every day under capacity limits, driver shifts, and customer time windows (e.g., a restaurant that accepts deliveries only between 9 and 11 a.m.). A route that is short on a map can arrive before a window opens and wait, or arrive after it closes and fail, and using fewer vehicles saves cost while making windows harder to meet. The dispatcher needs routes that serve every customer within its window with as little travel as possible.

Files in the data package

File	Contents
`depots.csv`	The single depot and its plotting coordinates.
`customers.csv`	One row per customer with demand, service time, time window, and plotting coordinates.
`vehicles.csv`	One row per vehicle with capacity and shift start and end.
`travel_times.csv`	Travel time in minutes for every ordered pair of locations.

Required constraints

1. Every customer is served exactly once.
2. Each vehicle makes at most one route, leaving the depot at its shift start and returning by its shift end. Unused vehicles are allowed.
3. The total demand on a route does not exceed the vehicle's capacity.
4. Service starts no earlier than the window start and no later than the window end. A vehicle that arrives early waits, and service may finish after the window end.
5. A vehicle stays at a customer for the full service time and departs when service ends, so its arrival at the next stop is that departure plus the travel time.

Required output

A `routes` list with one entry per used vehicle containing `vehicle_id`, `visits` (customer identifiers in route order), `arrivals` and `service_starts` (the time at each visit, in route order), and `return_time`.

Measures

The primary measure is total travel time across all routes, which excludes waiting and service. The secondary measure is vehicles used (vehicles serving at least one customer), and every comparison also reports computation time.

Rule to beat

Nearest feasible neighbor, one vehicle at a time. Take vehicles in identifier order. Each vehicle leaves the depot at its shift start. Repeatedly move to the nearest unserved customer (by travel time from the current location, ties by customer identifier) that fits the remaining capacity, can start service by its window end, and still allows a

return to the depot by the shift end. When no customer qualifies, the vehicle returns and the next vehicle starts. If every vehicle has returned and a customer is still unserved, the rule reports "no feasible decision found".

Method directions and reading

Method 1 is the rule to beat. A researched method may adapt the Clarke–Wright Savings method from class to check windows before each merge, use a time-window insertion heuristic (Solomon, 1987, *Operations Research*) or improve routes by relocating customers, exchanging customers between routes, or reversing route segments, with feasibility checked after every move.

Stress test

The uncertain input is travel times (e.g., a congestion factor per scenario) or service times. The fixed-plan execution rule follows each planned route in order and counts every service that starts after its window end and every late return as a constraint violation.

Scope boundary

Mapping and navigation are outside the required scope. Every customer is required, and choosing which customers to skip belongs to option 7.

Checked example

Applying the rule to beat to the hand-checkable case gives a total travel time of 352.417 minutes. The decisions and every recomputed measure appear in `worked-example/` in the data ZIP, with full precision in `metrics.json`. Reproducing this value to three decimals is a useful first test of a team's data loading, checker, and rule.

Where the rest of the project is defined

The project guide (`team-project-guide.pdf`) defines what every decision tool must do, the milestones, the grading rubric, the contents of this data package, and the exact rules the instructor's checker applies. This brief adds only what is specific to option 3.